

Soul-Driven Agent: An Information-Retrieval Agent with Reflective, Hand-Editable Memory

May 22, 2026

1 Project links

- **GitHub repository:** <https://github.com/Tiansve/Luna-IR-Agent>
- **Demo video:** https://drive.google.com/file/d/18IVicd0BkaRg-KlZ9YlnuNEk93u7zA_X/view?usp=sharing

2 Motivation and contribution

This project is directly inspired by OpenClaw, which exposes an agent to a set of tools through a gateway but treats the agent’s memory as a single session cache. The interesting question I wanted to push on was whether memory itself could be the primary IR target — not just a buffer for external retrieval, but a first-class store that the agent reads, writes, and reflects on. A standard RAG agent retrieves passages from a static corpus and writes nothing back; I wanted an agent in which every conversation becomes retrievable evidence, repeated patterns get distilled into typed insights, and stable insights are promoted into a human-readable “Soul” document that ships into every system prompt.

Building on the OpenClaw starting point, this project contributes:

- A three-layer memory hierarchy (episodic / semantic / Soul) with distinct timescales and persistence formats.
- A two-stage reflection engine: an extractor that compresses recent episodes into insight operations, and an updater that promotes stable insights into the Soul, with code-level guard-rails rather than prompt-level pleas.
- A Soul that is a Markdown / YAML document, not an opaque vector, so the user can read it, edit it, and roll it back across versioned snapshots.
- A first-class write-side IR tool (`update_soul_note`) that lets the agent append to a whitelist of soft Soul fields without going through reflection, so that direct memory updates and slow reflective updates coexist as two complementary write paths.

3 System design

3.1 The three memory layers

Episodic memory stores one record per past turn (query, tool calls, final answer) in JSONL on disk, with a lazily-built cosine matrix for similarity recall. **Semantic memory** holds typed insights of the form (`content`, `category`, `confidence`, `supporting_episodes`). Confidence moves asymmetrically: +0.10 on reinforcement, −0.15 on contradiction, auto-dropped below 0.15 unless already promoted. The asymmetry encodes the prior that user feedback should outweigh the agent’s own generalisations, while the gap between the dropping threshold and the floor prevents a single bad turn from blowing away a hard-earned insight. The **Soul** layer is a structured Markdown document with fields *identity*, *values*, *knowledge_about_user*, *learned_patterns*, and *open_questions*, plus a structured *evolution_log*.

At each turn, the context builder assembles: the Soul (always), top-*K* insights and top-*K* episode digests retrieved by semantic similarity, and the five tool schemas.

3.2 Reflection engine

Triggered automatically every `REFLECT_EVERY` unreflected episodes (default 3) or manually via `/reflect`. Two LLM calls in strict JSON mode:

- **Extractor.** Given all existing insights plus the unreflected episode batch, the model proposes *new*, *reinforced*, and *weakened* insight ids. Any operation referencing an unknown id is a silent no-op. Malformed JSON skips the pass; the live agent is unaffected.
- **Updater.** Runs only when promotion candidates exist (confidence ≥ 0.80 and ≥ 2 supporting episodes). The model proposes *additions* and *removals* on the four reflection-writable Soul fields. Guards enforced in code, not in prompt: *open_questions* is reserved for the human or for the `update_soul_note` action, each list is capped at 15 entries, at most two removals per pass, every removal requires a non-empty rationale, additions must reference a candidate insight id, and the old Soul is snapshotted to `soul_history/vN.yaml` before any write.

3.3 Tools, including action-as-IR

The model sees five tools through the standard OpenAI tool-use protocol. Three are read-side IR (`search_docs`, `recall_episodes`, `recall_insights`), one is external IR (`web_search` via Brave), and one is **write-side IR**: `update_soul_note` lets the agent append to *open_questions* or *knowledge_about_user* directly. Identity, values, and

learned patterns are deliberately *not* writable through this tool — durable persona changes are required to go through the slow, vetted promotion path.

3.4 Engineering robustness

The embedding subsystem (`memory/embed.py`) is a single façade with a remote backend probed once for dimension and a deterministic hash fallback of dimension 512. A sticky-disable flag avoids per-call retry-then-fail latency after one remote failure. Embedding requests are batched, with a recursive halving retry path that finally truncates a single offending input — a real failure mode on Berget’s 512-token embedding model. The doc-RAG index is fingerprinted by file metadata, chunking parameters, and model name, cached in `doc_index.npz`, and rebuilt automatically when any input changes. On a corpus of ~2500 chunks, this reduces startup from ~230 s (first build) to ~2 s (warm cache).

4 Behaviour and evaluation

What worked. Snapshot-before-write made the system safe to iterate on — I could let reflection run aggressively in development and roll back the Soul without losing episodic or insight data. The Markdown form of the Soul made debugging interpretable in a way vector memory would not have been: I could literally read what the agent thought it had learned. The two-path write design — fast direct writes via `update_soul_note` for soft fields, slow reflective promotion for durable persona changes — kept the system both responsive to explicit user preferences and resistant to noisy single-turn signals.

Limitations. Reflection quality is bounded by the JSON-mode reliability of the chosen model; on a weaker chat model the extractor occasionally proposes insights with no episode support, which the code-level guards downgrade to no-ops but which still waste a reflection call. The promotion threshold (0.80 confidence, ≥ 2 supports) is currently fixed rather than calibrated. Single-user only; multi-tenant would require keying every store path by user id.

5 Reflection on using AI tools

I used **Claude Code** throughout the project, but in two quite different modes.

Where AI assistance helped clearly. The boilerplate-heavy parts — the OpenAI-compatible client with batching and retry, the YAML/Markdown round-trip for the Soul, the JSONL stores, the slash-command dispatcher, the smoke test — were faster to draft with Claude Code than by hand, and the diffs were short enough to audit per change. The architectural shape of the project (three memory layers, snapshot-before-write, code-level guards instead of prompt-level guards) came out of a back-and-forth design discussion before any code was written; treating the model as a design partner for that conversation was more valuable than as a code generator.

Where I had to overrule it. The first reflection-updater drafts relied on prompt-level instructions like “do not delete more than two entries.” I had to push back and move those into Python — gating a write path with a sentence in a prompt is exactly the failure mode an LLM-driven system should avoid. Similarly, an initial draft of the embedding façade caught remote failures per call; this was technically correct but produced a 60 s wait every turn after the remote backend went down. The sticky-disable flag was a correctness fix that required reading the actual behaviour, not the code.

What I take away. AI coding tools accelerate the parts of a project where the spec is clear and the surface area is small. They are less reliable at decisions that span files — where guards should live, what should be in the prompt vs. in the code, how failure modes compound across modules. Verifying every non-trivial change against the actual runtime was non-negotiable.